

# Cloud Computing

Practical Viva — Complete Q&A; Guide

---

| # | Practical                     | Topics Covered                               |
|---|-------------------------------|----------------------------------------------|
| 1 | Create an EC2 Instance        | AWS, AMI, Instance Types, Key Pair, RDP, IIS |
| 2 | Google App Engine             | GCP, App Engine, Cloud Shell, Deployment     |
| 3 | Salesforce Apex Programming   | Apex Class, Email, Developer Console, Logs   |
| 4 | Salesforce Custom Application | Custom Objects, Fields, Tabs, LMS App        |

Prepared for Viva Examination | Simple & Precise Language

# PRACTICAL 1: Create an EC2 Instance on AWS

## Part A: Core Concepts

### Q: What is AWS (Amazon Web Services)?

A: AWS is a cloud computing platform provided by Amazon. It offers services like computing power (EC2), storage (S3), databases (RDS), and much more — all over the internet. You pay only for what you use, like a utility bill.

### Q: What is EC2?

A: EC2 stands for Elastic Compute Cloud. It is a service that lets you rent virtual machines (called instances) on the cloud. You can choose the OS, size, and configuration you need. It is 'elastic' because you can scale up or down based on demand.

### Q: What is a Virtual Machine (VM)?

A: A virtual machine is a software-based computer that runs on a physical computer. It behaves exactly like a real machine — has its own OS, CPU, RAM, and storage — but it is created and managed by software. Multiple VMs can run on one physical server.

### Q: What is an AMI (Amazon Machine Image)?

A: An AMI is a pre-built template/blueprint used to launch an EC2 instance. It contains the operating system (like Windows or Linux), software, and configuration. In this practical, we used Microsoft Windows Server 2022 Datacenter AMI (ami-038b0fc5213087d0).

### Q: What Instance Type was used and why?

A: We used t3.micro. It has 2 vCPUs and 1 GB RAM. It is free-tier eligible, meaning new AWS users can use it free for up to 750 hours/month. The 't' stands for 'burstable', meaning it can temporarily use more CPU when needed.

### Q: What is a Key Pair and why is it needed?

A: A Key Pair is a pair of cryptographic keys — a Public Key stored by AWS and a Private Key (.pem file) downloaded by you. For Windows EC2, you upload this .pem file to decrypt and retrieve the Administrator password. Without it, you cannot log into your instance.

### Q: What is a Security Group?

A: A Security Group is a virtual firewall that controls inbound and outbound traffic for your instance. For a Windows server, you need to allow RDP (port 3389) traffic so you can remotely connect. By default, it allows all outbound traffic.

### Q: What is RDP and what port does it use?

A: RDP stands for Remote Desktop Protocol. It allows you to remotely control a Windows machine over a network by displaying its graphical desktop. It uses port 3389. On Mac/Linux you use a separate RDP client; on Windows you use the built-in Remote Desktop Connection.

### Q: What is a Public IPv4 address?

A: It is the internet-facing IP address assigned to your EC2 instance. You use this address to connect to your instance from anywhere on the internet via RDP. Note: this address changes every time you stop and restart the instance (unless you use an Elastic IP).

### Q: What is a Public DNS?

A: Public DNS (Domain Name System) is a human-readable address mapped to your instance's Public IP. Example: ec2-65-0-72-55.ap-south-1.compute.amazonaws.com. You can use it instead of the IP

address to connect to your instance.

### Q: What is the difference between Stop and Terminate?

A: Stop = pause the instance (like shutting down a PC). Data is saved. You are not billed for compute, but storage billing continues. Terminate = permanently delete the instance and its data. Cannot be undone.

### Q: What AWS Region was used?

A: Asia Pacific (Mumbai) — ap-south-1. AWS Regions are geographic locations where AWS has data centers. Choosing a nearby region reduces network latency.

## Part B: IIS Web Server (Steps After Connecting)

### Q: What is IIS?

A: IIS stands for Internet Information Services. It is Microsoft's web server software built into Windows Server. It allows your server to host websites and serve web pages over HTTP/HTTPS. It is similar to Apache or Nginx on Linux.

### Q: How did you install IIS on the EC2 instance?

A: After connecting via RDP, we opened Server Manager → clicked 'Add Roles and Features' → selected 'Role-based installation' → selected the server → checked 'Web Server (IIS)' → clicked Install. IIS was installed successfully.

### Q: What is wwwroot?

A: wwwroot is the default root folder for IIS websites. Its full path is C:\inetpub\wwwroot. Any HTML/web files you place here are served by IIS when someone visits your server's IP or DNS in a browser.

### Q: What HTML code did you create for testing?

A: We created an index.html file in C:\inetpub\wwwroot with this content:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Awantika's First HTML Page</title>
  </head>
  <body>
    <h1>Welcome to My Website</h1>
    <p>This is a paragraph of text on my first web page.</p>
    <a href='https://www.example.com'>Visit Example.com</a>
  </body>
</html>
```

### Q: How do you verify IIS is working?

A: Open a browser inside the EC2 instance and go to: File → C:\inetpub\wwwroot\index.html. Or from outside, open the instance's Public IP in a browser. You should see your HTML page with 'Welcome to My Website'.

### Q: What are the steps to create an EC2 instance in brief?

1. Log in to AWS Console → Go to EC2 → Click 'Launch Instance'
2. Enter a name (e.g., Awantika's Server)
3. Choose AMI: Windows Server 2022
4. Choose Instance Type: t3.micro
5. Create a Key Pair (RSA, .pem format) → Download it
6. Configure Security Group: Allow RDP (port 3389)

7. Set Storage: 30 GB (default for Windows)
8. Click 'Launch Instance'
9. Wait for instance to start → Get Password using .pem file
10. Connect via RDP using Public DNS + Administrator + Password

**Q: What is Availability Zone?**

A: An Availability Zone (AZ) is an isolated data center within a Region. In our practical, ap-south-1a was used. Multiple AZs in a region provide redundancy — if one data center fails, others continue working.

**Q: What is Free Tier in AWS?**

A: AWS Free Tier allows new users to use certain services for free for 12 months. For EC2, it includes 750 hours/month of t3.micro or t2.micro instances. Beyond that, charges apply.

## PRACTICAL 2: Installation and Configure Google App Engine

### Part A: Google Cloud Platform (GCP) Basics

#### Q: What is Google Cloud Platform (GCP)?

A: GCP is Google's cloud computing platform that provides services like computing, storage, databases, machine learning, and networking. It runs on the same infrastructure that powers Google Search, YouTube, and Gmail.

#### Q: What is Google App Engine?

A: Google App Engine (GAE) is a Platform-as-a-Service (PaaS) offered by GCP. It allows developers to deploy and run applications without managing the underlying servers or infrastructure. App Engine automatically handles scaling, load balancing, and server maintenance.

#### Q: What is PaaS?

A: PaaS stands for Platform as a Service. The cloud provider manages the infrastructure (servers, OS, runtime), and you just write and deploy your code/application. App Engine is PaaS. Compare: IaaS (like EC2) = you manage OS. SaaS (like Gmail) = fully managed, you just use the app.

#### Q: What is a GCP Project?

A: A Project in GCP is the basic organizational unit. All resources (App Engine apps, APIs, billing) belong to a project. In our practical, we created a project named 'Awantika's Project' with Project ID: awantikas-project. The Project ID is unique and permanent — cannot be changed later.

#### Q: What is Cloud Shell?

A: Cloud Shell is a browser-based command-line terminal provided by GCP. It comes pre-installed with tools like gcloud, git, python, etc. You can run commands directly in GCP without installing anything on your local machine.

#### Q: What is App Engine Admin API?

A: The App Engine Admin API is a Google API that must be enabled before you can deploy applications to App Engine. It provisions and manages App Engine applications. We enabled it from: GCP Console → Search 'App Engine Admin API' → Enable.

#### Q: What is GitHub and why was it used here?

A: GitHub is a cloud-based platform for storing and managing code using Git version control. In this practical, we cloned a GitHub repository (inventory-management-system) into Cloud Shell using: `git clone`. This brought the project code into the GCP environment.

#### Q: What command was used to clone the GitHub repository?

A: `git clone https://github.com/Awantika-Mudaliar/inventory-management-system.git`

★ *Note: If the directory already exists, git clone fails. Navigate into it using: `cd inventory-management-system`*

#### Q: What is hello.py and what did it output?

A: `hello.py` is a simple Python script created for testing App Engine. When run with: `python hello.py`, it printed: 'Hello from Google App Engine using GitHub'. This confirms that Python runs correctly in Cloud Shell and the project is set up properly.

#### Q: What is the gcloud command to set a project?

```
gcloud config set project [PROJECT_ID]
```

A: This sets the active GCP project for all subsequent gcloud commands in the session.

**Q: What files are needed to deploy an app on App Engine?**

A: Two main files are required: (1) app.yaml — configuration file that tells App Engine the runtime (e.g., python39, php), environment, and other settings. (2) Main application file — e.g., main.py for Python or index.php for PHP.

**Q: What command deploys an app to App Engine?**

```
gcloud app deploy
```

A: This command reads app.yaml and deploys your application to App Engine. After deployment, you can view it with: gcloud app browse

## Part B: App Engine Architecture

**Q: How does App Engine handle scaling?**

A: App Engine automatically scales the number of instances based on incoming traffic. When traffic increases, it starts more instances. When traffic decreases, it scales down. This is called Auto Scaling and is a key advantage over traditional servers.

**Q: What is the difference between Standard and Flexible Environment in App Engine?**

A: Standard Environment: Runs in a sandboxed environment, scales to zero (no cost when idle), supports specific runtimes (Python, Java, PHP, Go, Node.js). Flexible Environment: Runs in Docker containers, more customizable, does not scale to zero, supports any language.

**Q: What is the Organisation in GCP project creation?**

A: An Organisation in GCP is the top-level resource, usually linked to a Google Workspace domain (like isquareit.edu.in). Projects are created under an organisation for better management and access control.

**Q: What are the benefits of using App Engine over EC2?**

A: App Engine (PaaS) requires no server management — no OS updates, no patching. EC2 (IaaS) gives more control but you manage the OS and software yourself. For deploying apps quickly, App Engine is faster. For custom server configurations, EC2 is better.

# PRACTICAL 3: Creating an Application in Salesforce using Apex

## Part A: Salesforce and Apex Basics

### Q: What is Salesforce?

A: Salesforce is the world's #1 cloud-based CRM (Customer Relationship Management) platform. It helps businesses manage customer data, sales, marketing, and support — all in the cloud. It is a SaaS (Software as a Service) product.

### Q: What is CRM?

A: CRM stands for Customer Relationship Management. It is a system that helps companies track interactions with customers, manage sales pipelines, send emails, and analyze business data. Salesforce is the most popular CRM platform globally.

### Q: What is Apex in Salesforce?

A: Apex is Salesforce's own proprietary programming language. It is similar to Java in syntax. It runs on Salesforce servers (not on local machine) and is used to write business logic, automate tasks, send emails, and interact with Salesforce data (called sObjects).

### Q: What type of language is Apex?

A: Apex is a strongly-typed, object-oriented programming language. It is case-insensitive, compiled, and runs on the Salesforce platform. It supports classes, interfaces, triggers, exceptions, and collections like Java.

### Q: What is the Developer Console in Salesforce?

A: The Developer Console is the built-in IDE (code editor) in Salesforce. You access it from: Setup Menu (gear icon) → Developer Console. You can write, save, and run Apex classes and anonymous code, view logs, run SOQL queries, and debug code — all inside the browser.

### Q: What is a Salesforce Developer Edition?

A: Salesforce Developer Edition is a free, full-featured version of Salesforce available for developers to learn and build apps. It has some limitations on data storage and users but includes all major features including Apex, Lightning, and custom objects.

## Part B: The Apex Email Code

### Q: Write the Apex class code used in this practical.

A: The Apex class 'first' contains a static method 'sendMail' that sends an email:

```
public class first {  
    public static void sendMail(String address, String subject, String body) {  
        Messaging.SingleEmailMessage mail =  
            new Messaging.SingleEmailMessage();  
        String[] ToAddresses = new String[] { address };  
        mail.setToAddresses(ToAddresses);  
        mail.setSubject(subject);  
        mail.setPlainTextBody(body);  
        Messaging.sendEmail(  
            new Messaging.SingleEmailMessage[] { mail }  
        );  
    }  
}
```

### Q: What does the 'public class first' mean?

A: 'public' is the access modifier — it means this class is accessible from anywhere in the Salesforce org. 'class' keyword defines a class. 'first' is the name of the class. Every Apex class must be saved with the same name as the class.

#### Q: What does 'public static void' mean?

A: 'public' = accessible from anywhere. 'static' = the method belongs to the class itself, not to an instance/object. So you can call it as `first.sendMail()` without creating an object. 'void' = the method does not return any value.

#### Q: What are the three parameters of sendMail?

A: 1. address (String) — The recipient's email address. 2. subject (String) — The subject/title of the email. 3. body (String) — The main content/body of the email.

#### Q: What is Messaging.SingleEmailMessage?

A: It is a built-in Salesforce class that represents a single email message. You create an object of this class, set its properties (To address, subject, body), and then send it. It is part of the Messaging namespace provided by Salesforce.

#### Q: Explain each line of the sendMail method.

```
→ Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
```

Creates a new email message object called 'mail'.

```
→ String[] ToAddresses = new String[] { address };
```

Creates an array of strings containing the recipient email address.

```
→ mail.setToAddresses(ToAddresses);
```

Sets the 'To' field of the email — who will receive it.

```
→ mail.setSubject(subject);
```

Sets the subject line of the email.

```
→ mail.setPlainTextBody(body);
```

Sets the body/content of the email as plain text (not HTML).

```
→ Messaging.sendEmail(new Messaging.SingleEmailMessage[] { mail });
```

Actually sends the email. `sendEmail()` takes an array of email messages.

#### Q: How did you run the Apex code?

A: In Developer Console → Debug → Open Execute Anonymous Window → Type the following code and click Execute:

```
first.sendMail('test@example.com', 'Hello', 'This is a test email from Apex');
```

#### Q: What is 'Execute Anonymous'?

A: Execute Anonymous is a feature in the Developer Console that lets you run Apex code immediately without saving it as a class. It is used for testing and debugging. The code runs once and its output appears in the Execution Log.

#### Q: What is an Execution Log?

A: The Execution Log is shown at the bottom of the Developer Console after running code. It shows events like `USER_DEBUG` messages, method calls (`METHOD_ENTRY`), memory allocation (`HEAP_ALLOCATE`), and errors. It helps you debug your Apex code.

#### Q: What log event confirms the email was sent / code ran correctly?

A: In the Execution Log, you look for the Status = 'Success' in the Logs tab. If you used `System.debug()` inside the method, you would see a `USER_DEBUG` event with your message. A successful execution



with no errors confirms the email was sent.

**Q: What is the difference between `setPlainTextBody` and `setHtmlBody`?**

A: `setPlainTextBody()` sends the email as plain text — no formatting, no HTML tags. `setHtmlBody()` allows you to send HTML-formatted emails with bold, links, colors, etc. In our code, we used `setPlainTextBody` for simplicity.

**Q: What is `Messaging.sendEmail()` and what does it take as input?**

A: `Messaging.sendEmail()` is the method that actually sends the email. It takes an array of `Messaging.SingleEmailMessage` objects. Even if you are sending just one email, you still pass it inside an array.

**Q: How do you create a new Apex class in Developer Console?**

A: Developer Console → File → New → Apex Class → Enter class name 'first' → Click OK. The editor opens with a basic class skeleton. You write your code and press Ctrl+S (or File → Save) to save and compile.

**Q: What happens if there is a syntax error in your Apex code?**

A: Salesforce compiles Apex code when you save it. If there is a syntax error, it shows a red error message at the bottom of the Developer Console. The code will not be saved until the error is fixed. This is different from some languages where errors only appear at runtime.

**Q: What is `System.debug()` and how is it used?**

A: `System.debug()` is like `print()` in Python or `System.out.println()` in Java. It prints a message to the Execution Log. Example: `System.debug('Email sent!');` This helps you trace code flow and check values during debugging.

## PRACTICAL 4: Custom Application using Salesforce Cloud (Mini Project — LMS)

### Part A: Salesforce Data Model Concepts

#### Q: What is a Custom Object in Salesforce?

A: A Custom Object is a database table you create in Salesforce to store data specific to your business. Salesforce has Standard Objects like Account, Contact, Lead — but if you need to store Student data, Book data, etc., you create Custom Objects. Custom objects have names ending in \_\_c in the API (e.g., Student\_\_c).

#### Q: What is a Custom Field?

A: A Custom Field is a column you add to a Standard or Custom Object to store specific data. For example, the Student object has custom fields: Student Name (Text), Class (Text), Roll Number (Number). Field names also end in \_\_c in API (e.g., Roll\_Number\_\_c).

#### Q: What is an API Name in Salesforce?

A: API Name is the technical name used when accessing objects/fields via code (Apex, SOQL) or API. It is different from the Label (which is user-friendly). Example: Label = 'Roll Number', API Name = 'Roll\_Number\_\_c'. Custom fields always have \_\_c suffix.

#### Q: What is a Record Name in Salesforce?

A: Every Salesforce object has a Record Name field — it is the primary identifier of each record (like a primary key label). For the Student object, we set Record Name = 'Student Name' with Text data type. For Account, it is 'Account Name'.

#### Q: What are the Data Types used for fields in this practical?

A: Text: Stores alphanumeric characters. Used for Student Name, Class, Author Name, Publisher, Book Name. Number: Stores numeric values (no decimal). Used for Roll Number — Length: 5, Decimal Places: 0.

#### Q: What is Deployment Status?

A: Deployment Status in Salesforce has two options: 'In Development' — the object is visible only to admins for testing. 'Deployed' — the object is visible to all users. We selected 'Deployed' so users can access the Student and Read Book objects.

### Part B: Objects Created in the LMS App

#### Q: What custom objects were created for the LMS (Library Management System)?

A: Two custom objects were created: 1. Student — to store student information. 2. Read Book — to store book information.

#### Q: What fields were created for the Student object?

| Field Label  | API Name       | Data Type   | Details                          |
|--------------|----------------|-------------|----------------------------------|
| Student Name | Name           | Text(80)    | Record Name field — auto-created |
| Class        | Class__c       | Text(10)    | Stores class like TE, BE, SE     |
| Roll Number  | Roll_Number__c | Number(5,0) | Length 5, Decimal 0              |

#### Q: What fields were created for the Read Book object?

| Field Label | API Name       | Data Type              |
|-------------|----------------|------------------------|
| Book Name   | Name           | Text(80) — Record Name |
| Author Name | Author_Name__c | Text(50)               |
| Publisher   | Publisher__c   | Text(50)               |

## Part C: Tabs and App Setup

### Q: What is a Tab in Salesforce?

A: A Tab is a link in the navigation bar that takes users to a specific Object or Page. For custom objects, you need to manually create a Custom Object Tab to make it accessible in the app. Without a tab, users cannot navigate to your custom object.

### Q: How did you create tabs for the custom objects?

A: Setup → Search 'Tabs' → Under Custom Object Tabs → Click 'New' → Select Object (Student or Read Book) → Choose a Tab Style (icon) → Click Next → Save. We created two tabs: 'Students' (Computer icon) and 'Read\_Books' (Books icon).

### Q: What is Tab Style?

A: Tab Style is the visual icon/color that represents the tab in the navigation bar. For example, we used 'Computer' icon for Students tab and 'Books' icon for Read\_Books tab. This helps users visually identify different sections of the app.

### Q: What is App Manager / Lightning Experience App Manager?

A: App Manager is where you create and manage Lightning Apps in Salesforce. A Lightning App is a collection of tabs and objects grouped together for a specific purpose. We created an app called 'LMS' (Library Management System) using App Manager.

### Q: What is a Lightning App?

A: A Lightning App is a customized navigation experience in Salesforce Lightning Experience. It groups specific tabs (like Students, Read\_Books) together so users see only what they need. Each app can have its own name, icon, tabs, and user profile access.

### Q: What steps did you follow to create the LMS Lightning App?

1. Setup → Search 'App Manager' → Click 'New Lightning App'
2. Enter App Name: LMS, Developer Name: LMS → Click Next
3. App Options: Keep defaults (Standard navigation, Desktop and Phone) → Next
4. Utility Items: Skip → Next
5. Navigation Items: Search 'Read' → Add Read\_Books and Students → Next
6. User Profiles: Add 'System Administrator' → Click Save & Finish
7. App appears in App Launcher → Click LMS → Students and Read\_Books tabs are visible

### Q: What is App Launcher in Salesforce?

A: App Launcher is the grid icon (■) at the top left of Salesforce. It shows all available apps. You can search and switch between apps from here. After creating the LMS app, we found it in App Launcher and launched it.

### Q: What is a User Profile in Salesforce?

A: A User Profile controls what a user can see and do in Salesforce. It sets permissions for objects, fields, apps, etc. When creating the LMS app, we assigned the 'System Administrator' profile so only

admins can access the app.

**Q: How did you add a new Student record?**

A: In the LMS app → Click 'Students' tab → Click 'New' button → Fill in: Student Name = AA, Class = TE, Roll Number = 36 → Click Save. The student record was created and displayed with all the fields including Created By and Last Modified By.

## Part D: Object Manager and Setup

**Q: What is Object Manager in Salesforce?**

A: Object Manager (Setup → Object Manager) is the central place to view and manage all Standard and Custom Objects in your Salesforce org. You can see their fields, page layouts, buttons, and relationships from here.

**Q: What are Page Layouts in Salesforce?**

A: Page Layouts control how fields are arranged on a record's detail and edit page. When you create a custom field, you choose which Page Layout to add it to. For example, all Student fields were added to the 'Student Layout'.

**Q: What is the difference between Standard and Custom Objects?**

A: Standard Objects come pre-built with Salesforce: Account, Contact, Lead, Opportunity, etc. Custom Objects are created by you for your specific needs: Student, Read Book, etc. Custom objects have \_\_c in their API names.

**Q: What are Optional Features when creating a Custom Object?**

A: When creating a custom object, optional features include: Allow Reports (use object data in reports), Allow Activities (log calls/emails against records), Track Field History (see history of changes), Allow in Chatter Groups, Enable Licensing. In the Student object, we enabled Allow Reports and Allow Activities.

**Q: What is Object Classification (Allow Sharing, Bulk API, Streaming API)?**

A: Allow Sharing: Enables record-level sharing (who can see which records). Allow Bulk API Access: Allows mass data operations via API. Allow Streaming API Access: Enables real-time notifications when data changes. All three were enabled for both Student and Read Book objects.

**Q: What is SOQL?**

A: SOQL stands for Salesforce Object Query Language. It is similar to SQL but used to query Salesforce data. Example: `SELECT Id, Name, Class__c FROM Student__c`. You can run SOQL queries in the Developer Console's Query Editor tab.

**Q: What is the full flow of creating a mini project in Salesforce?**

Step 1: Create Custom Objects (Student, Read Book) — Object Manager → New Custom Object

Step 2: Create Custom Fields for each object — Object Manager → Object → Fields & Relationships → New

Step 3: Create Custom Object Tabs — Setup → Tabs → New Custom Object Tab

Step 4: Create a Lightning App — App Manager → New Lightning App → Add tabs → Assign profiles

Step 5: Add Records — Go to App → Tab → New → Fill data → Save

Step 6: Verify — Check that records are saved with all fields visible correctly

---

## Quick Comparison Cheat Sheet

| Term                         | Practical  | Meaning                                      |
|------------------------------|------------|----------------------------------------------|
| EC2                          | AWS        | Virtual Machine on AWS cloud                 |
| AMI                          | AWS        | OS template to launch instance               |
| Key Pair (.pem)              | AWS        | Security key to access instance              |
| Security Group               | AWS        | Firewall rules for instance                  |
| RDP (port 3389)              | AWS        | Remote Desktop Protocol for Windows          |
| IIS                          | AWS        | Microsoft's web server (like Apache)         |
| wwwroot                      | AWS        | IIS default web folder: C:\inetpub\wwwroot   |
| App Engine                   | GCP        | PaaS to deploy apps without managing servers |
| Cloud Shell                  | GCP        | Browser-based terminal in GCP                |
| gcloud                       | GCP        | Command-line tool for GCP                    |
| app.yaml                     | GCP        | App Engine configuration file                |
| Apex                         | Salesforce | Salesforce's Java-like programming language  |
| Developer Console            | Salesforce | Browser-based IDE for Apex                   |
| Messaging.SingleEmailMessage | Salesforce | Apex class to compose email                  |
| Messaging.sendEmail()        | Salesforce | Apex method to send email                    |
| Custom Object (__c)          | Salesforce | User-defined database table                  |
| Custom Field (__c)           | Salesforce | User-defined column in object                |
| Tab                          | Salesforce | Navigation link to an object                 |
| Lightning App                | Salesforce | Collection of tabs for a purpose             |
| App Manager                  | Salesforce | Tool to create Lightning Apps                |
| Object Manager               | Salesforce | Manage all objects and their fields          |
| SOQL                         | Salesforce | Query language for Salesforce data           |
| PaaS                         | General    | Platform as a Service                        |
| IaaS                         | General    | Infrastructure as a Service                  |
| SaaS                         | General    | Software as a Service                        |

Good Luck for Your Viva! You've Got This! ■